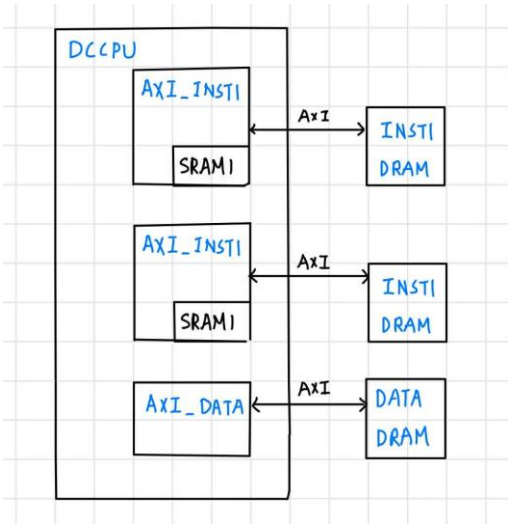
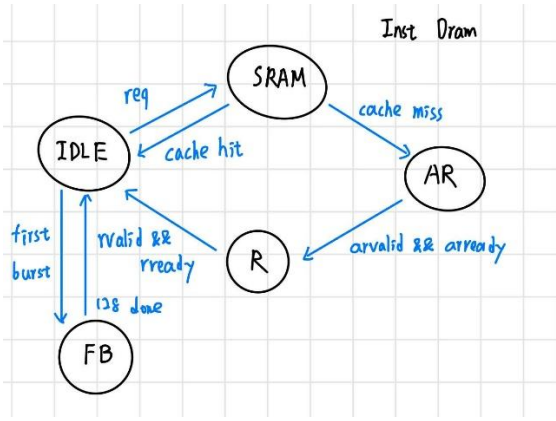
IC LAB Midterm Project Report

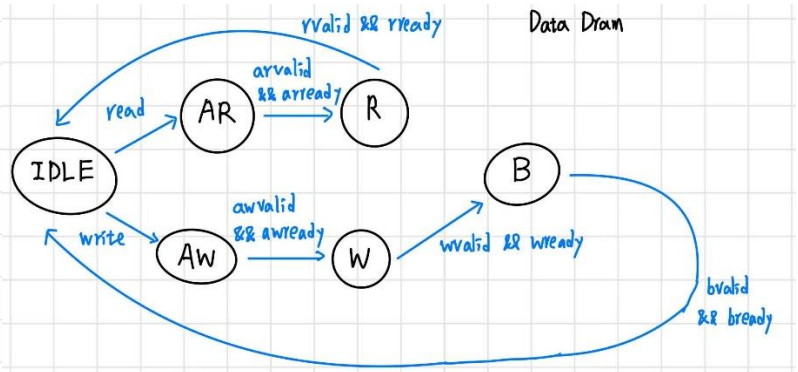
Block diagram:



1. **INST_DRAM FSM:**



2. **DATA_DRAM FSM:**



我把上面兩個分別寫成 submodule，運作模式:

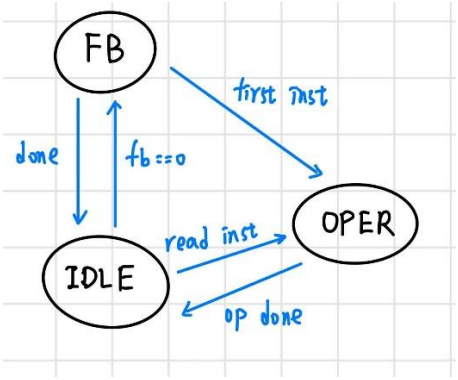
- **AXI_INST submodule:**

  我在這個 submodule 內用 SRAM 當作 cache，我 SRAM 大小開 128*22，我第一次會 burst 128 個 instructions 到 SRAM。之後就是發送 require_valid 訊號到 submodule，然後去 SRAM 看有沒有 cache hit，如果有 hit 就不用去 DRAM 讀了。如果 cache miss 就去 DRAM 讀，但是 miss 後就很難繼續 hit，所以必須往後 burst 讀新的 instructions，我最終是選擇往後 burst 32 個 instructions，當然如果往後 burst 128 個會讓後續 cache hit 機率提高，但是如果測資一直 jump 就會需要一直讀新的 burst 128，這樣反而有點浪費時間，於是我在 trade-off 後選擇 burst 32 個。

- **AXI_DATA submodule:**

  這個 submodule 就比較正常，就是 AXI 之間的互動，要注意的就是兩個 core 要共用這個 DATA_DRAM，所以我有用到 id 這個訊號，當我是 core 1 在使用的時候 id = 4'b0001，如果是 core 2 在使用 id = 4'b0010，這樣當我收到 rdata 的時候就可以依據 rid 去分辨這個 rdata 是要給 core 1 還是 core 2。

3. Main function FSM:



- **流程:**

  最一開始要給 first burst 指令到 AXI_INST submodule 裡面，接著就開始拿到 instruction 後去 OPER state 運算，兩個 core 共用 ALU，我會讓 core 1 優先去做，除非 core 2 是 store。

  這樣可以確保同時遇到 load/store 的時候可以有效的優先處理 store 指令，而且共用 ALU 可以少開一個乘法器，進而減少面積。因為我覺得跟 DRAM 溝通的 latency 就很長了，core 1 和 core 2 平行運算沒有討到太多的便宜。

- **MULT**

  在這個 project critical path 最長的運算一定是乘法。

  路徑: register 8 取 1 取出 rs, rt → 16 bit * 16 bit 的乘法 → register 8 取 1 存回 rd, rl

  如果整個流程一個 cycle 做完會太長，所以我切成三份，就是我箭頭隔開那樣，這樣一個 cycle 只做乘法就可以把 cycle time 壓低，也可以降低 slack 的壓力。

- **Stall signal**

  我看 pdf 上面有說當 stall is low 的時候會檢查 all registers of two cores，因為不確定助教那邊的 pattern 長得如何，所以我保險一點就是讓 stall1, stall2 同時降下來，如下圖所示。

```
// stall
always @(*)
begin
    next_stall_1 = 1; next_stall_2 = 1;

    if(op_done1 && op_done2)
    begin
        next_stall_1 = 0;
        next_stall_2 = 0;
    end
end
```

  因為 pdf 上面的規定有很多不確定性，所以我就是兩個 core 都做完後才會去 require 下一個 instruction，如果 performance 要好一點只能祈禱 cache hit 多發生點吧。

4. Generate Pattern

這次助教有說測資都不應該讓運算結果 overflow，這就非常考驗生測資的能力，我的實力還不足

以讓我手寫一堆組合語言測資，我是用 random 產 instruction 並且對數值做限制，這雖然可以避免 overflow 但 instruction 根本沒有邏輯，因此後續也需要借他人的 pattern 測試。

- **How to avoid overflow**

  測資發生 overflow 有兩個情況，分別是:

  (1) R-type 運算的結果超出 16 bit 範圍

  (2) Dram access address 超過範圍

- **避免(1) overflow**

  我想到的方法就是運用 ADD 這個 operation，我的 r5 都是存 15 這個數字，我每幾個 instruction 就會強制生成 ADD 指令，隨機選一個 register 和 r5 做 ADD，這樣可以把該 register 的數值限制在 15 以內，這樣就比較不會出現 overflow 的狀態，但如果 MULT 連續執行有很大的機率會 overflow，所以我的 instruction 每隔 3 個就會生成 ADD，讓 MULT 連續執行的機率降低。

  我的 register 也有做限制:

  (1) R0: 永遠都是 0，不會執行任何 instruction

  (2) R1-R4: rs, rt, rd 都從這幾個裡面選，ADD 也是從這四個裡面依序選擇跟 r5 ADD

  (3) R5: 數值只會給 15 (一開始 pc = 0 會執行 load 到 r5 的指令，在這之前 r5 是 0)

  (4) R6, R7: load 和 store 的 address 運算會用到

```
for (c = 1; c <= 2; c = c + 1) begin
    for (pc_idx = 0; pc_idx < 4096; pc_idx = pc_idx + 1) begin
        g_op = $urandom_range(0, 7);
        // r1-r4 operate
        g_rs = $urandom_range(1, 4);
        g_rt = $urandom_range(1, 4);
        g_rd = $urandom_range(1, 4);
        g_func = $urandom_range(0, 7);
        g_imm = $urandom_range(0, 15);

        if (pc_idx == 0) begin
            g_op = 3'b100; // LOAD
            g_rs = 0;      // r0
            g_rt = 5;      // r5
            g_imm = 0;
        end
        else if (pc_idx % 3 == 0) begin
            g_op = 3'b000;       // R-Type
            g_func = 3'b010;     // AND
            g_rs = 5;            // r5 (Mask = 15)

            // r1, r2, r3, r4
            if (pc_idx % 12 == 0) g_rt = 1;
            else if (pc_idx % 12 == 3) g_rt = 2;
            else if (pc_idx % 12 == 6) g_rt = 3;
            else g_rt = 4;

            g_rd = g_rt; // AND r1 r1 r5...
        end
```

- **避免(2) overflow**

  1. Constrain load/store address

  Address = $(rs + imm) * 2 + offset$

  要避免 address 超範圍就要控制好 rs 和 imm 的大小，我的 rs 只會從 R0, R6, R7 這三個去選，其中 R0 必定是 0，而 R6, R7 就要看他們從 DATA_DRAM 拿到什麼值，我的 DATA_DRAM 前 15 個 address 就是設計給 R6, R7 load 的(range: 16-2000)，imm 也有設範圍(16-63)。這個設計的缺點就是我-不能測到邊界的 address，所以後續還是需要借他人的 pattern 測試。

```
// ================================================================
// 1. Generate Data DRAM
// ================================================================
for (i = 0; i < 4096; i = i + 1) // 1000-2fff, total 8192 bytes -> 4096 lines
begin
    if (i == 0) begin
        // [Index 0] r5 Mask value: 0x000F (15)
        $fdisplay(fd_d, "@%05X", 32'h1000 + i * 2);
        $fdisplay(fd_d, "0F 00");
    end
    else if (i < 16) begin
        // [Index 1~15] r6, r7 Safe Base Addresses (16 ~ 2000)
        safe_base = $urandom_range(16, 2000);
        $fdisplay(fd_d, "@%05X", 32'h1000 + i * 2);
        $fdisplay(fd_d, "%02X %02X", safe_base[7:0], safe_base[15:8]);
    end
    else begin
        // [Index 16~2047] random (-31 ~ +31)
        small_data = $urandom_range(0, 31);
        if ($urandom_range(0, 1))
            small_data = -small_data;
        $fdisplay(fd_d, "@%05X", 32'h1000 + i * 2);
        $fdisplay(fd_d, "%02X %02X", small_data[7:0], small_data[15:8]);
    end
end
```

```
// Constraint: LOAD / STORE
if (g_op == 3'b100 || g_op == 3'b101) begin
    // r0, r6, r7
    base_choice = $urandom_range(0, 2);
    if (base_choice == 0)
        g_rs = 0;
    else if (base_choice == 1)
        g_rs = 6;
    else
        g_rs = 7;

    g_imm = $urandom_range(16, 63);

    if (g_op == 3'b100) begin
        init_choice = $urandom_range(0, 9);
        if (init_choice == 0) begin
            g_rs = 0; g_rt = 5; g_imm = 0; // load mask to r5
        end
        else if (init_choice == 1) begin
            g_rs = 0; g_rt = 6; g_imm = $urandom_range(1, 15); // load Base to r6
        end
        else if (init_choice == 2) begin
            g_rs = 0; g_rt = 7; g_imm = $urandom_range(1, 15); // load Base to r7
        end
    end
end
```

2. Constrain branch/jump pc range

要先提到 spec 說兩個 core 不會同時都是 store 指令，所以我 core1 store 只出現在 pc 是奇數，core2 store 只出現在 pc 是偶數，然後每次Δpc都要是奇數(因為要維持兩個 core 的奇偶性)，所以 branch/jump 的Δpc必須是奇數，然後每次的Δpc範圍都是-32 到 31 那附近，如果發現這樣會超過範圍就要修正Δpc直到不會超過範圍。

```
// Constraint: Jump & Branch (delta must be odd)
if (g_op == 3'b110) begin // BEQ
    s_imm_int = ($urandom_range(0, 31) * 2) - 32; // -32 to 30, imm must be even
    if ($signed(pc_idx + 1) + s_imm_int < 0)
    begin
        s_imm_int = s_imm_int + 32;
    end
    if ($signed(pc_idx + 1) + s_imm_int > 4095)
    begin
        s_imm_int = s_imm_int - 32;
    end
    g_imm = s_imm_int[6:0];
end
else if (g_op == 3'b111) begin // Jump
    target_pc = pc_idx + ($urandom_range(0, 31) - 16) * 2 + 1; // -31 to 31, must be odd
    if (target_pc < 0)
    begin
        target_pc = target_pc + 32;
    end
    if (target_pc > 4095)
    begin
        target_pc = target_pc - 32;
    end
    g_j_addr = target_pc * 2;
end
```

我有寫第二版 PATTERN，就是讓兩個 core 一起跑到 pc=4095 然後結束模擬。

為了好寫，指令就是不往回跳了，因為無限迴圈真的很難讓他跳出去，直接控制 jump 和 beq 只能往前走，並且手動增加 load/store 同時發生的情況，去確認先後執行有沒有問題。我就是看哪個 core 會先到 4090 那邊，另一個就直接 jump 到那裡，並且讓 pc=4095 不要執行 jump 或是 beq，雖然這個方法有點搞笑，但還是成功在兩個 core 相同 ic 的情況下到達 pc=4095。